CoAP Diagnostic Message Notation
A textual notation for CoAP interactions

Jaime Jimenez, **Carsten Bormann**

The Problem

- CoAP drafts/RFCs notate exchanges in many ways: split vs composed URIs, `Header: /Payload:` vs `Req: /Res:` vs nothing
- No clear convention, more like variations on a scheme
- No accepted **machine-checkable** format
  ➳ malformed CBOR/JSON examples may slip through.

The Notation

- **First line**: (request) method + composed URI,
  or response code + reason phrase
- **Options**: `Name: value` pairs (RFC 9203 style)
- **Content-Format**: code point **and** media type
  ➽ tells tooling which validator to use
- **Payload**: EDN for CBOR, JSON, CoRE Link Format, or hex
  (to do: RFC 8710 application/multipart-core)
- Blank line separates options from payload (HTTP style)

Example

Request

```
POST /ps
Content-Format: TBD606 (application/core-pubsub+cbor)

{
  / topic-name /
  0: "living-room-sensor",
  / resource-type /
  2: "core.ps.data"
}
```

Response

```
2.01 Created
Location-Path: /ps/h9392
Content-Format: TBD606 (application/core-pubsub+cbor)

{
  / topic-name /
  0: "living-room-sensor",
  / topic-data /
  1: "/ps/data/1bd0d6d",
  / resource-type /
  2: "core.ps.data"
}
```

Why It Matters

- **Parse ➳ identify format ➳ dispatch validator ➳ report errors with line numbers**
- Similar to prototypical checking in **kramdown-rfc**; optionally pull in **CDDL** for schema-level checks
- Purely documentary, bad input is highlighted in the doc build
- One notation across specs, guides, and teaching material

Questions to the WG

- Is it OK to optimize for **application** specifications?
  What else do we need when going into CoAP guts?
- Is the Internet-Draft **clear**?
- Please try **applying** it to examples in your documents.
  Does that work out well?
- Get **prototype** checker in place.
  Does that still work well?
- After a round of this feedback,
  decide whether to **adopt** this spec for Informational.